

✖ Étape 1 — Analyse & Conception du projet

Objectif : comprendre et structurer l'ensemble du projet avant d'écrire la moindre ligne de code. C'est une **étape critique** pour éviter de coder à l'aveugle.

✖ 1.1 — Définition des fonctionnalités principales

Qu'est-ce que l'application doit permettre de faire ?

◆ Fonctionnalités utilisateurs :

- Créer un compte / se connecter
- Créer des projets (ex : "Application mobile", "Site Web")
- Ajouter des tâches dans chaque projet
- Attribuer une tâche à un utilisateur
- Modifier ou supprimer une tâche
- Marquer une tâche comme terminée
- Filtrer les tâches (par état, utilisateur, priorité, date)
- Voir une vue d'ensemble des tâches par projet

◆ Fonctionnalités système :

- Gestion des utilisateurs (authentification)
 - Communication entre l'API, le front-end Web et l'app Qt
 - Mode hors ligne dans l'app Qt (stockage local)
 - Synchronisation automatique avec résolution de conflits
-

✖ 1.2 — Types d'utilisateurs

Tu peux commencer simplement avec un seul type d'utilisateur :

- **Utilisateur standard** : peut créer et gérer ses propres projets/tâches

(Optionnel plus tard : rôles "admin", "invité", etc.)

✖ 1.3 — Cahier des charges fonctionnel simplifié

Élément	Description
---------	-------------

Nom	TaskSync – gestionnaire de tâches collaboratif
-----	--

Utilisateurs	Inscription, connexion, gestion de session
--------------	--

Tâches	CRUD (Create, Read, Update, Delete), avec titre, description, état, dates
--------	---

Projets	Contiennent des tâches, partagés ou personnels
---------	--

Interfaces	Web (HTML/CSS/TS), App Qt (C++), API REST (Python/Java)
------------	---

Objectifs	Simplicité d'usage, efficacité, travail multi-plateforme
-----------	--

1.4 — Cas d'utilisation (UML)

Voici les **principaux cas d'utilisation** que tu peux représenter dans un diagramme :

- Créer un compte
- Se connecter
- Créer un projet
- Ajouter une tâche
- Modifier une tâche
- Supprimer une tâche
- Afficher les tâches
- Synchroniser avec le serveur

Acteur principal : Utilisateur

1.5 — Diagramme de classes (UML simplifié)

Voici les entités principales avec quelques attributs :

◆ Utilisateur

- id
- nom
- email
- mot_de_passe (hashé)

◆ **Projet**

- id
- nom
- description
- date_creation
- propriétaire (Utilisateur)

◆ **Tâche**

- id
- titre
- description
- état (à faire, en cours, terminée)
- date_limite
- date_creation
- assignée_a (Utilisateur)
- projet (Projet)

Tu peux créer un diagramme avec ces classes et leurs relations (composition, association).

📌 **1.6 — Conception de l'API REST (routes principales)**

Méthode	Endpoint	Description
POST	/auth/register	Créer un nouveau compte
POST	/auth/login	Se connecter
GET	/projects	Lister les projets de l'utilisateur
POST	/projects	Créer un projet
GET	/projects/{id}	Détails d'un projet
DELETE	/projects/{id}	Supprimer un projet
POST	/tasks	Ajouter une tâche

Méthode	Endpoint	Description
GET	/tasks	Lister toutes les tâches
PUT	/tasks/{id}	Modifier une tâche
DELETE	/tasks/{id}	Supprimer une tâche

Tu peux enrichir ensuite avec : filtres, pagination, etc.

1.7 — Maquettes simples des interfaces

Pas besoin de logiciel compliqué : fais des croquis rapides pour :

- L'écran de **connexion / inscription**
- L'interface principale Web : liste des projets, liste des tâches
- L'interface Qt : vue compacte des tâches, boutons d'action, synchronisation

 Tu peux les faire à la main ou utiliser un outil comme [Excalidraw](#), [Figma](#), [Whimsical](#), etc.

Estimation du temps pour l'étape 1

Tâche	Temps estimé
Définition des fonctionnalités	1 h
Cahier des charges	1 h
Cas d'utilisation UML	1 h
Diagramme de classes UML	2 h
Conception API REST	2–3 h
Croquis UI (maquettes)	1–2 h
Total	6 à 10 h

Souhaites-tu que je t'aide à générer les schémas UML, les routes d'API en OpenAPI, ou les maquettes de l'interface une fois que tu auras fait ton premier brouillon ?

Vous avez dit :

C'est quoi les schéma UML?

ChatGPT a dit :

Très bonne question ! Voici une explication claire et concrète :

C'est quoi les schémas UML ?

UML signifie **Unified Modeling Language** (langage de modélisation unifié).

C'est un langage visuel utilisé pour **modéliser un projet logiciel** avant (et parfois pendant) le développement.

Les **schémas UML (ou diagrammes UML)** permettent de **représenter visuellement les éléments clés d'un système** :

- les **acteurs** (utilisateurs, systèmes)
- les **relations entre objets/classes**
- les **comportements** attendus
- les **interactions**

Les 2 types de diagrammes UML les plus utiles pour toi dans ce projet :

1. **Diagramme de cas d'utilisation (Use Case)**

Que peuvent faire les utilisateurs dans l'application ?

- Il représente les **actions** possibles (se connecter, créer un projet, etc.)
- Il montre **qui fait quoi** (acteur → action)

 Exemple :

scss

CopierModifier

[Utilisateur] —▶ (Créer un projet)

└─▶ (Ajouter une tâche)

└─▶ (Modifier une tâche)

 Utile pour : comprendre **les fonctionnalités à implémenter** du point de vue de l'utilisateur.

2. Diagramme de classes

Quelles sont les **entités du système**, leurs **attributs**, et comment elles sont liées ?

- Chaque **classe** = une entité (ex : Utilisateur, Projet, Tâche)
- On y indique :
 - Les **attributs** (ex : nom, email, date_creation)
 - Les **relations** (ex : "un projet contient plusieurs tâches")

 Exemple simplifié :

markdown

CopierModifier

Utilisateur

- id

- nom

- email

Projet

- id

- nom

- description

- propriétaire : Utilisateur

Tâche

- id

- titre

- état

- projet : Projet

- assignée_a : Utilisateur

✖ Utile pour : **construire ta base de données**, ton **API REST**, et comprendre la structure de ton app.